

Definir el stack tecnológico oficial de KODA POS.

Aquí debes pensar no solo en “qué funciona”, sino en:

- escalabilidad
- costos
- mantenimiento
- velocidad de desarrollo
- seguridad
- offline
- móvil
- SaaS

Voy a darte una arquitectura moderna y REALISTA para un SaaS POS profesional.

STACK TECNOLÓGICO OFICIAL RECOMENDADO PARA KODA POS

KODA POS — Stack Tecnológico Oficial

1. Frontend Web

Framework principal

- Next.js

Librería base

- React

Lenguaje

- TypeScript

Estilos

- Tailwind CSS

Componentes UI

- shadcn/ui

Manejo de estado

- Zustand

Tablas avanzadas

- TanStack Table

Formularios

- React Hook Form

Validaciones

- Zod

Gráficas y estadísticas

- Recharts

Animaciones

- Framer Motion

Cliente HTTP

- Axios

Manejo de caché y requests

- TanStack Query

2. Aplicación móvil

Framework recomendado

- React Native

Plataforma de desarrollo

- Expo

Compatibilidad

- Android
- iOS

Ventajas

- Un solo código base
 - Integración sencilla con React
 - Publicación en App Store y Play Store
-

3. Backend

Framework principal

- NestJS

Runtime

- Node.js

Lenguaje

- TypeScript

Arquitectura

- Modular
 - Multi-tenant
 - API REST
 - WebSockets
-

4. Base de datos

Motor principal

- PostgreSQL

ORM

- Prisma ORM

Seguridad avanzada

- PostgreSQL Row Level Security (RLS)
-

5. Sistema Offline

Tecnologías

- IndexedDB
- Dexie.js
- Service Workers
- PWA

Objetivo

- Permitir ventas sin internet
 - Sincronización automática posterior
-

6. Tiempo Real

Librería

- Socket.IO

Uso

- Actualización en tiempo real

- Inventario
 - Ventas
 - Notificaciones
-

7. Autenticación

Sistema recomendado

- Auth.js (NextAuth)

Métodos de login

- Google OAuth
- Email y contraseña
- Posible login con Microsoft

Seguridad

- JWT
 - Refresh Tokens
 - Session Management
-

8. Registro con Google

Proveedor

- Google Cloud Console

Configuración necesaria

- OAuth Consent Screen
 - OAuth Client ID
 - OAuth Secret
-

9. Pagos y suscripciones

Pasarela principal

- Mercado Pago

Funciones

- Suscripciones
- Pagos mensuales
- Webhooks
- Renovaciones automáticas

SDK

- Mercado Pago SDK
-

10. Hosting Frontend

Recomendado

- Vercel

Ventajas

- Optimizado para Next.js
 - Deploy automático
 - CDN integrado
-

11. Hosting Backend

Recomendado inicialmente

- Railway

Alternativas

- Render
- DigitalOcean
- AWS

12. Hosting Base de Datos

Recomendado

- Neon PostgreSQL

Alternativas

- Supabase
 - Railway PostgreSQL
-

13. Almacenamiento de archivos

Recomendado

- Cloudinary

Uso

- Logos
 - Imágenes de productos
 - Archivos
-

14. Emails del sistema

Servicio recomendado

- Resend

Funciones

- Verificación de email
- Recuperación de contraseña
- Alertas
- Notificaciones

15. Logs y monitoreo

Logs

- Winston
- Pino

Monitoreo

- Sentry
-

16. Seguridad

Protección

- Helmet
 - Rate Limiting
 - CORS
 - CSRF Protection
 - XSS Protection
-

17. Contenedores y DevOps

Contenedores

- Docker

Control de versiones

- GitHub

CI/CD

- GitHub Actions

18. Caché y rendimiento

Tecnología

- Redis

Uso

- Caché
 - Sesiones
 - Cola de trabajos
 - Rate limiting
-

19. Colas y trabajos en segundo plano

Tecnología

- BullMQ

Uso

- Sincronización offline
 - Emails
 - Procesamiento masivo
 - Reportes
-

20. Documentación API

Herramienta

- Swagger
-

21. Hardware POS

Compatibilidad

- Impresoras térmicas ESC/POS
 - Lectores de código de barras
 - Gavetas de dinero
 - Pantallas táctiles
-

22. Analítica y métricas

Herramientas

- PostHog
- Google Analytics

Uso

- Actividad de usuarios
 - Conversión
 - Métricas SaaS
-

23. Dominio y DNS

Proveedores recomendados

- Cloudflare
 - Namecheap
-

24. CDN y protección

Recomendado

- Cloudflare

Funciones

- CDN
 - SSL
 - Protección DDoS
 - Caché
-

25. Arquitectura General

Frontend

- Next.js
- Tailwind
- Zustand

Backend

- NestJS
- Prisma
- PostgreSQL

Infraestructura

- Vercel
- Railway
- Cloudflare

Servicios externos

- Mercado Pago
- Google OAuth
- Resend
- Cloudinary

Seguridad

- JWT
- RLS
- Redis
- Rate limiting

COSAS IMPORTANTES QUE AÚN SE TE ESCAPAN

Y esto es MUY importante porque muchos proyectos fallan aquí.

1. Sistema de sincronización offline

Esto será probablemente:

la parte MÁS difícil de KODA POS.

Debes investigar:

- conflictos
- colas
- sincronización
- duplicados
- consistencia

2. Arquitectura multi-tenant

Debes decidir:

- shared DB
- DB por cliente
- híbrido

3. Manejo de inventario concurrente

Ejemplo:

2 cajas venden el mismo producto.

Debes definir:

- locks
- stock negativo
- reconciliación

4. Manejo de suscripciones SaaS

Debes crear:

- planes
 - estados
 - renovación
 - grace period
 - suspensión
-

5. Sistema de backups

MUY importante.

Debes definir:

- backups automáticos
 - restauración
 - snapshots
-

6. Sistema de auditoría

Registrar:

- ventas eliminadas
 - cambios
 - accesos
 - cierres
-

7. Migraciones de base de datos

MUY importante para SaaS.

Con Prisma:

- migrations
 - versionado
-

8. Observabilidad

Debes monitorear:

- errores
 - rendimiento
 - caídas
 - uso de CPU
 - consultas lentas
-

9. Estrategia mobile

Debes decidir:

¿La app móvil será?

Administrativa

o

POS completo

Porque eso cambia muchísimo.

10. Estrategia de costos

MUY importante.

Debes calcular:

- hosting
 - DB
 - almacenamiento
 - emails
 - imágenes
 - tráfico
-

Lo MÁS IMPORTANTE ahora

Antes de programar:
haz primero esto:

ORDEN CORRECTO

- 1. Arquitectura SaaS**
- 2. Modelo multi-tenant**
- 3. Modelo offline**
- 4. Base de datos**
- 5. Roles y permisos**
- 6. Flujos POS**
- 7. Wireframes/UI**
- 8. Infraestructura**
- 9. DevOps**
- 10. Desarrollo**

Porque si empiezas a programar sin eso:
vas a rehacer MUCHAS cosas después.